

QVerse — Week 16: Capstone Sprint II — Validate, Explain, and Present

Theory Booklet | Introduction to Quantum Computing & Programming

QVerse

2026

How this booklet fits the course

This theory booklet is designed to be read alongside `16_capstone_validate_present.ipynb`. It contains the conceptual and mathematical material that should be explained during Week 16; the notebook then turns those ideas into experiments, circuit construction, and exercises.

Learning objectives

- Validate expected versus observed behavior.
- Improve reproducibility.
- Communicate limitations honestly.
- Prepare a concise technical presentation.

Recommended workflow

1. Read the relevant theory section.
2. Predict the result of the corresponding notebook experiment.
3. Run the notebook.
4. Explain any difference between the prediction and the output.
5. Complete the core exercises without treating the solution notebook as a substitute for reasoning.

1. From prototype to evidence

Week 15 created a working prototype. Week 16 asks a harder question:

Does the evidence actually support the conclusion?

The final phase should improve validity, reproducibility, and communication rather than adding unnecessary features.

A finished project is not simply code that runs. It is a documented technical argument.

2. Validation hierarchy

Validation can occur at several levels.

Unit or identity checks

Does a small component implement a known transformation?

Analytical checks

Does the result match a formula or exact solution?

Simulator cross-checks

Does a statevector result agree with sampled behavior?

Baseline comparison

Does the quantum circuit outperform or differ from the chosen baseline in the expected way?

Noise or hardware comparison

How robust is the result once ideal assumptions are relaxed?

The strongest projects use more than one form of validation.

3. Expected versus observed

Every important result should be presented as a comparison.

Expected

State the prediction numerically or qualitatively.

Observed

Report the actual measurement, expectation value, fidelity, or distribution.

Agreement

Quantify the discrepancy when possible.

For a scalar result,

$$\Delta = x_{\text{obs}} - x_{\text{exp}}.$$

A normalized error can be

$$\epsilon = \frac{|x_{\text{obs}} - x_{\text{exp}}|}{|x_{\text{exp}}|}.$$

For probabilities, statistical uncertainty should also be considered.

4. Finite-shot uncertainty

If a probability is estimated from N shots,

$$\hat{p} = \frac{K}{N},$$

with approximate standard error

$$\sigma_{\hat{p}} \approx \sqrt{\frac{\hat{p}(1 - \hat{p})}{N}}.$$

Therefore a discrepancy smaller than normal sampling variation may not be meaningful.

If the result is obtained from real hardware, sampling uncertainty and hardware error should be discussed separately.

5. Reproducibility audit

A new student should be able to reproduce the project without asking the author for hidden information.

Check:

- Is the entry point obvious?
- Are dependencies listed?

- Are versions recorded?
- Are installation instructions complete?
- Are data-generation steps included?
- Are shot counts specified?
- Are random seeds fixed where useful?
- Is backend information recorded?
- Is bit ordering explained when relevant?
- Are plots labeled?
- Are expected results stated?

Reproducibility is a technical property of the project, not merely a documentation preference.

6. Hardware metadata

If hardware is used, record enough context to make the result interpretable:

- provider/backend;
- date;
- number of shots;
- transpilation settings;
- optimization level;
- physical qubit/layout information if relevant;
- calibration information if readily available.

Hardware results can drift over time. The final report should not imply that one observed histogram is timeless.

7. Peer review

Peer review is useful because authors become accustomed to their own assumptions and conventions.

A reviewer should score:

- correctness;
- reproducibility;
- understanding of the quantum concept;
- experimental reasoning;
- code quality;
- communication.

The most useful comment is specific and actionable.

Weak:

The explanation is confusing.

Better:

The Grover section says the oracle increases probability, but the oracle only changes phase; the diffuser is what converts that phase marking into amplitude amplification.

8. Code quality for a teaching project

The code does not need production-scale architecture, but it should have:

- descriptive variable names;
- short reusable functions where appropriate;
- minimal duplicated code;
- comments that explain *why*, not every syntax token;
- deterministic setup where useful;
- tests for critical helper functions;
- no unexplained hidden state in notebook cells.

A notebook should run from top to bottom in a fresh environment.

9. Result presentation

A strong figure or table should answer the research question immediately.

Every plot needs:

- labeled axes;
- units when relevant;
- meaningful title or caption;
- legend when multiple series exist;
- enough context to understand what is being compared.

Avoid showing many nearly redundant plots. One clear quantitative figure is better than several decorative outputs.

10. The 5-8 minute presentation

A concise structure is:

1. **Problem** - one sentence.
2. **Quantum idea** - one core concept.
3. **Circuit/workflow** - only the essential diagram.
4. **Prediction** - what theory says.
5. **Result** - one main plot or table.
6. **Interpretation** - why it happened.
7. **Limitations** - what the result does not prove.
8. **Extension** - one sensible next step.

The presentation should be understandable to a student who completed approximately the first half of the course.

11. How to explain the “quantum” part

A final project should contain a direct answer to:

What is genuinely quantum about this project?

Good answers identify a mechanism.

Example for Grover:

The oracle encodes the target as a relative phase and the diffuser uses interference to amplify the marked amplitude.

Example for teleportation:

The protocol uses a pre-shared entangled state and Bell-basis processing so an unknown input state can be reconstructed remotely after two classical correction bits are communicated.

Example for VQE:

A parameterized quantum state is prepared and quantum expectation values of Hamiltonian terms define the objective optimized by a classical routine.

12. Limitations

Limitations increase credibility when stated precisely.

Possible limitations:

- only a small simulated system was tested;
- the noise model is simplified;
- only one backend run was performed;
- no quantum advantage was established;
- the oracle cost was not included;
- the ansatz may not represent the true optimum;
- finite shots limit statistical precision;

- hardware calibration varied.

A limitation should identify what inference cannot be made from the evidence.

13. Failure analysis

If observed and expected results disagree, investigate systematically.

Possible causes include:

1. bit-order convention;
2. incorrect target state;
3. missing inverse or uncomputation;
4. insufficient shots;
5. random sampling;
6. incorrect measurement basis;
7. transpilation effects;
8. noise;
9. stale notebook state;
10. version/API mismatch.

Do not immediately attribute every discrepancy to “quantum noise.”

14. Final technical summary

A concise technical summary of roughly 150-250 words should answer:

- What was the question?
- What quantum concept was used?
- What experiment was performed?
- What was the principal quantitative result?
- Did it match the prediction?
- What is the main limitation?

This summary can later become the core of a README, poster abstract, or project portfolio description.

15. Repository completion checklist

The final repository should contain:

- a clear README;
- setup/dependency instructions;
- the reproducible notebook or package;
- circuit diagrams;
- labeled results;
- written interpretation;
- references where appropriate;
- no private credentials or API tokens;
- no unexplained temporary outputs.

The repository itself is part of the deliverable.

16. Final standard of evidence

Before presenting, ask:

1. What exactly is my claim?
2. Which result supports it?
3. What theoretical prediction am I comparing against?
4. How large is the uncertainty?
5. What alternative explanation might exist?
6. Can another person reproduce the result?
7. What does this experiment *not* establish?

If these questions have explicit answers, the capstone has moved from “a circuit that runs” to a defensible technical experiment.

Suggested references

- IBM Quantum Learning, *Basics of Quantum Information* and current Qiskit learning material: <https://quantum.cloud.ibm.com/learning/en/courses>
- IBM Quantum Documentation and Qiskit guides: <https://quantum.cloud.ibm.com/docs>
- Quantum Rings, *Quantum 101*: <https://www.quantumrings.com/course/quantum101>
- M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, Cambridge University Press.

The software-facing details in the practical notebook should be treated as the canonical implementation for this course. The purpose of this booklet is to explain the physics, mathematics, and algorithmic reasoning behind that implementation.